Test Closure Report — Automation Testing

1. Introduction:

Project Name: MedusaJS E-Commerce Platform
Testing Type: Automation Functional & API Testing

2. Executive Summary:

Module/Area	Total test cases	Pass	Fail
User Registration, Authentication, Password Reset	**10**	**6**	**4**
Cart & Checkout & Payment	**11**	**7**	**4**
Customer Orders	**8**	**8**	**0**
Product Management	**7**	**7**	**0**

Overall Status: Fail – Retesting required after fixes and environment availability.

3. Objectives:

 Validate that all core automation test suites (registration, authentication, password management, cart, checkout, payment gateway, orders, reports, and product management) have been executed and meet the expected pass criteria.
 Confirm complete coverage of boundary conditions and input validations, including automated checks for invalid data, edge cases, and field constraints at both UI and API levels.
 Ensure consistent and accurate system behavior across UI automation and API tests, including correct error messages, validation responses, and data flow integrity.
 Review final test results, defect reports, and logs to ensure all critical issues are resolved or properly documented before closure.
 Validate data consistency across components (frontend, backend, and database) following automated execution of key workflows.
 Ensure all test artifacts are archived, including automated test scripts, API collections, execution reports, logs, and evidence.
 Document lessons learned and recommendations for improving test automation coverage, reliability, and efficiency in future cycles.

4. Key Observations / Findings (Automation + API Testing)

1. Validation & Functional Gaps

- Automated UI and API tests identified failures in **registration validation rules**, including phone number formatting, email pattern checks, name constraints, and password criteria.
- Checkout flow automation showed **inconsistent validation** for name, phone number, and postal code across UI and API responses.
- Product Management API tests revealed issues with **HTML/Unicode handling**, session failures, search endpoint inconsistencies, and import process errors, causing multiple automated scripts to fail.

2. UI Issues Detected via Automation

- Missing **error messages**, incorrect or absent **password strength indicators**, lack of **confirmation emails**, and missing dashboard sections caused automated UI tests to fail.
- Automated validation showed **catalog visibility issues**, broken elements, and inconsistent feedback messages on multiple screens.

3. System Behavior Issues

- API testing identified improper handling of **account lock/unlock flows**, missing or incorrect backend responses, and inconsistent **session expiry** behavior.
- Automated cart, order, and reporting scenarios frequently failed due to incorrect **cart badge counts**, non-functional **order filters**, and partially implemented **reporting modules**.

5. Recommendations (Automation + API Testing)

1. Validation & Functional Fixes

- Fix all core validation issues in **registration, authentication, password rules, checkout, product creation, and order management** across both UI and API endpoints.
- Enforce consistent **input formats**, **min/max field lengths**, and **validation rules** across UI, API responses, and database constraints.
- Align UI validation messages with API error responses to ensure consistent automation results.

2. UI Improvements

- Ensure proper display of **error messages**, **password strength indicators**, and **confirmation emails** to support automated feedback verification.
- Fix accessibility issues for key modules, including **product catalog**, **order filters**, **analytics dashboards**, and other pages required for end-to-end automation flows.

3. Environment / Access Fixes

- Grant full **Admin access to the Product Management module** to enable end-to-end automated testing for CRUD operations.
- Enable and stabilize **online payment flows** so both UI and API automation can validate gateway behavior, error handling, and edge cases.
- Ensure all required endpoints and test environments are deployed and synchronized to avoid frequent automation blocks.

4. Retesting

- Re-execute all previously **blocked, failed, or incomplete test cases** once the fixes are applied across UI, API, and backend services.
- Re-run the full automated regression suite after stabilizing modules to confirm system reliability and prevent reintroducing defects.

6. Metrics Overview:

Metric	Value
Total Test Cases	36
Pass	28
Fail	8
Test Pass %	77.77%
Test Fail %	22.22%
Critical / High Severity	10
Medium / Low Severity	26